

Advanced RAG: Graph-RAG Architecture

Reference Manual: Knowledge Graphs and Graph-Aware Inference

Sujeet Banerjee

Contents

1 Purpose and Scope	3
2 Core Data Representations	3
2.1 Document Chunks	3
2.2 Entities	3
2.3 Relationships	4
3 Ontology, Extraction Rulebook, and Graph Facts	4
3.1 Ontology or Schema	4
3.2 Extraction Rulebook	5
3.3 Graph Facts	5
3.4 Recommended Storage Separation	5
4 Ingestion Architecture	6
4.1 Vector Storage	6
4.2 Graph Storage	6
5 Should Chunks Be Enriched Before Vectorization?	7
5.1 Pattern A: Minimal Enrichment	7
5.2 Pattern B: Graph Metadata Enrichment	7
5.3 Pattern C: Embed Graph-Expanded Text	7
6 The Link Between VectorDB and GraphDB	8
7 Inference Architecture	8
8 Vector Retrieval Branch	9
9 Graph Retrieval Branch	9
10 Candidate Fusion	10
11 Candidate Scoring and Reranking	10
12 Graph-Guided Vector Retrieval	11
13 Retrieval Strategies	11
13.1 Vector-First	11
13.2 Graph-First	12
13.3 Parallel Hybrid Retrieval	12
14 Graph-Aware Context Construction	12

15 End-to-End Worked Example	13
15.1 Step 1: Chunking	13
15.2 Step 2: Entity Extraction	13
15.3 Step 3: Relationship Extraction	13
15.4 Step 4: Graph Validation	14
15.5 Step 5: Vectorization	14
15.6 Step 6: User Query	14
15.7 Step 7: Plain RAG Retrieval	14
15.8 Step 8: Graph-RAG Retrieval	14
16 Advantages Over Plain RAG	15
17 Reference Implementation Plan	15
17.1 Phase 1: Plain RAG Baseline	15
17.2 Phase 2: Entity and Relationship Extraction	15
17.3 Phase 3: Graph Storage and Provenance	16
17.4 Phase 4: Cross-Store Linkage	16
17.5 Phase 5: Parallel Retrieval	17
17.6 Phase 6: Candidate Reranking	17
17.7 Phase 7: Graph-Aware Context Builder	18
18 Evaluation Framework	18
19 Operational and Governance Considerations	19
20 Reference Architecture	20
21 Summary of Design Decisions	20

1 Purpose and Scope

Graph-RAG combines semantic retrieval from vector representations with explicit relationship retrieval from a graph-based knowledge representation. The architecture is intended to address questions whose answers depend not only on locating relevant passages, but also on identifying relationships, dependencies, paths, impacts, or multi-hop connections between entities.

A conventional Retrieval-Augmented Generation pipeline is commonly represented as:

Query → Embedding → Vector Search → Top-K Chunks → LLM

A Graph-RAG architecture introduces an additional knowledge representation:

Documents → Chunks → VectorDB
 Documents → Entities + Relationships → GraphDB

At inference time, the retrieval system may combine semantic relevance with graph structure:

Query → Vector Retrieval + Graph Retrieval → Candidate Fusion → Reranking →
 Context Construction → LLM

Graph-RAG is not a single fixed implementation pattern. The graph may influence retrieval before vector search, after vector search, in parallel with vector search, or primarily during context construction.

2 Core Data Representations

2.1 Document Chunks

A source document is divided into chunks suitable for embedding and retrieval.

Example:

Chunk C1:
 Order Service calls Payment Service.

Chunk C2:
 Payment Service uses Payments DB.

Chunk C3:
 Checkout depends on Order Service.

Each chunk normally receives a stable identifier:

```
chunk_id = C1
document_id = architecture_doc_01
```

The chunk identifier is important because it can provide provenance and linkage between vector evidence and graph entities.

2.2 Entities

Entities represent domain-specific objects.

Example entity types:

Service
 Database
 API
 Incident
 Team
 CustomerFeature

Example entities:

```
Order Service
Payment Service
Payments DB
Checkout
INC-102
```

Entities should normally have stable canonical identifiers:

```
service_order
service_payment
database_payments
feature_checkout
incident_102
```

2.3 Relationships

Relationships represent explicit domain connections.

Example:

```
Order Service --CALLS--> Payment Service
Payment Service --USES--> Payments DB
Order Service --SUPPORTS--> Checkout
INC-102 --AFFECTED--> Payment Service
```

The graph may be represented conceptually as:



3 Ontology, Extraction Rulebook, and Graph Facts

Three concepts should remain separate.

3.1 Ontology or Schema

The ontology defines what kinds of entities and relationships are valid.

Example:

```
Entities:
- Service
- Database
- Incident

Relationships:
- USES
- CALLS
- DEPENDS_ON
- AFFECTED
```

A schema may define valid source and target types:

```
Service --USES--> Database
```

The schema answers:

What entities and relationships are permitted in the knowledge model?

3.2 Extraction Rulebook

The extraction rulebook defines how text should be interpreted when identifying relationships.

Example:

USES:
 A Service uses a Database when the source explicitly states that the Service reads from, writes to, stores data in, retrieves data from, or persists data to the Database.

CALLS:
 A Service calls another Service when the source explicitly states that it invokes an API, endpoint, RPC interface, or synchronous service request.

The rulebook answers:

How should textual evidence be interpreted as a graph relationship?

3.3 Graph Facts

Graph facts are the extracted and validated results.

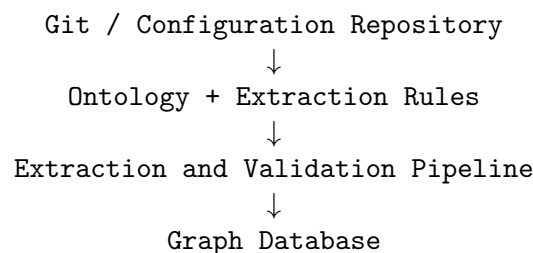
```
(Payment Service)-[:USES]->(Payments DB)
(Order Service)-[:CALLS]->(Payment Service)
```

The graph answers:

What relationships are represented as facts in the knowledge base?

3.4 Recommended Storage Separation

A practical architecture is:



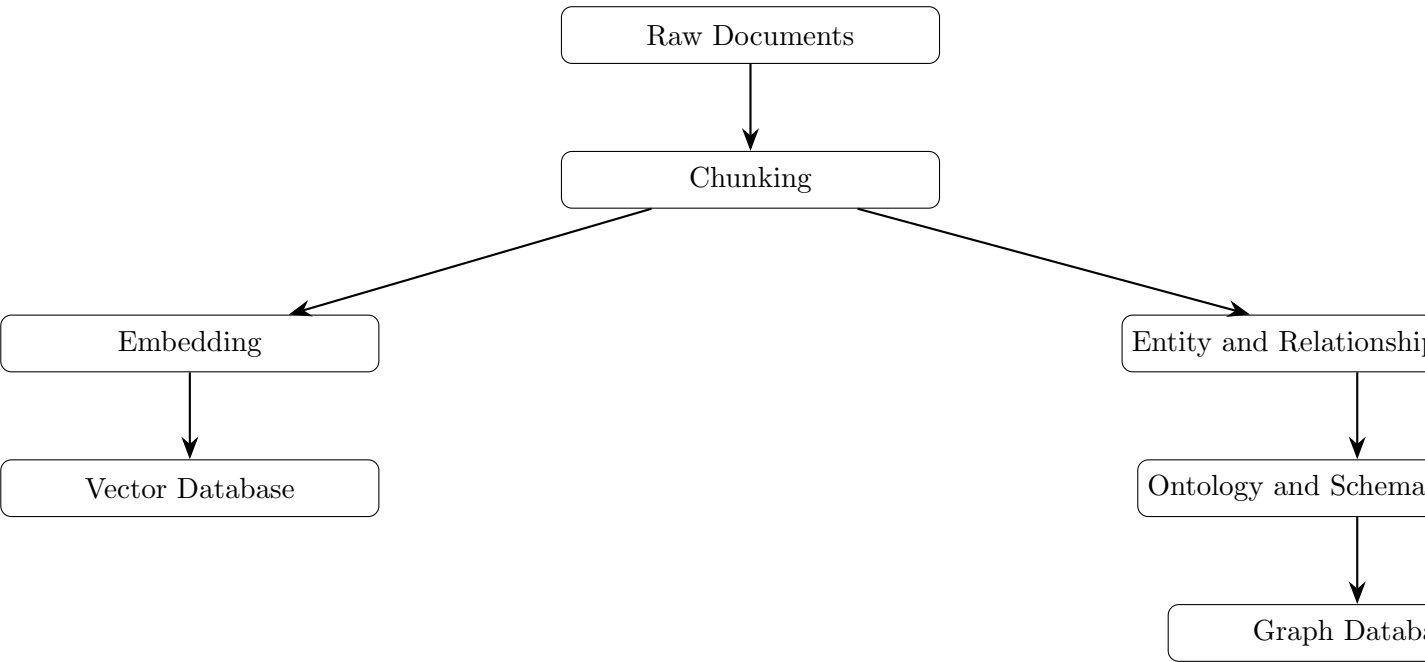
The GraphDB primarily stores extracted facts, entities, relationships, and provenance. The complete extraction rulebook does not need to be stored in the GraphDB.

A rule version may nevertheless be recorded as provenance:

```
(Payment Service)-[:USES {
  confidence: 0.94,
  extractor_version: "graph-extractor-v2",
  ontology_version: "1.3",
  rule_version: "2.1",
  source_chunk_id: "C2"
}]>(Payments DB)
```

4 Ingestion Architecture

A complete ingestion pipeline can be represented as:



4.1 Vector Storage

The original or normalized chunk text is embedded:

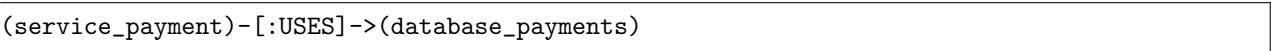


The vector store may contain:



4.2 Graph Storage

The graph extraction branch stores:



The same entity identifiers provide a logical bridge:

GraphDB	Shared Identifier	VectorDB
Payment Service	service_payment	Chunks mentioning service_payment
Payments DB	database_payments	Chunks mentioning database_payments

5 Should Chunks Be Enriched Before Vectorization?

There are several valid patterns.

5.1 Pattern A: Minimal Enrichment

The original chunk is embedded without graph expansion.

Example metadata:

```
{
  "chunk_id": "C2",
  "document_id": "architecture_doc_01",
  "entity_ids": [
    "service_payment",
    "database_payments"
  ]
}
```

Advantages include separation between local source evidence and global graph structure. A change to the graph does not automatically require regeneration of all affected embeddings.

This pattern is a suitable default implementation.

5.2 Pattern B: Graph Metadata Enrichment

Graph-derived information may be attached as metadata:

```
{
  "chunk_id": "C2",
  "entities": [
    "Payment Service",
    "Payments DB"
  ],
  "relationships": [
    "Payment Service USES Payments DB"
  ],
  "related_entities": [
    "Order Service",
    "Checkout"
  ]
}
```

The metadata can support filtering and candidate generation.

5.3 Pattern C: Embed Graph-Expanded Text

The original chunk may be augmented with additional graph-derived context before embedding.

Example:

```
Original:
Payment Service uses Payments DB.

Expanded:
Payment Service uses Payments DB.
Order Service calls Payment Service.
Order Service supports Checkout.
```

This approach should be used carefully. Graph-expanded text can blur the distinction between source evidence and derived knowledge. Graph changes may also require re-embedding previously generated vectors.

For most implementations, explicit graph traversal at retrieval time is preferable to indiscriminate expansion of chunk text before embedding.

6 The Link Between VectorDB and GraphDB

The relationship between graph evidence and vector evidence is usually not a relational database join based on vector similarity.

The common architecture is based on stable identifiers and provenance.

Example:

```
Graph:
(Payment Service {id: S1})
-[:USES]->
(Payments DB {id: D1})

Vector metadata:
Chunk C2
entity_ids = [S1, D1]
```

The retrieval layer can therefore answer:

Which chunks provide textual evidence for this graph node or graph path?

This may be implemented through:

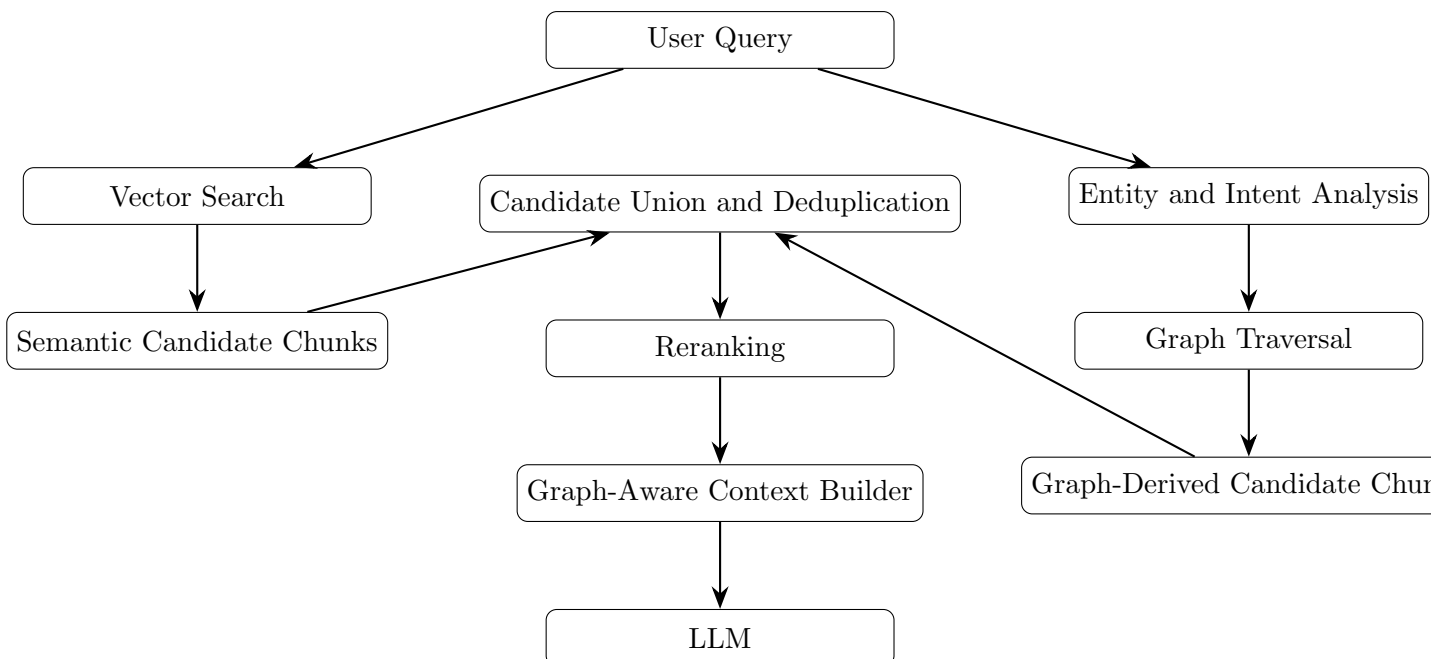
- entity IDs stored in vector metadata;
- chunk IDs stored as node or relationship provenance;
- an external entity-to-chunk index;
- a document provenance table;
- a unified retrieval system supporting both graph and vector operations.

7 Inference Architecture

Consider the query:

What happens to Checkout if Payments DB fails?

A hybrid inference architecture can perform parallel retrieval.



8 Vector Retrieval Branch

The query is embedded:

```
"What happens to Checkout if Payments DB fails?"
      |
      v
Query Embedding
      |
      v
Vector Search
```

Example results:

Chunk	Content	Similarity
C2	Payment Service uses Payments DB.	0.91
C8	A database outage caused payment authorization failures.	0.85
C12	Checkout requires successful payment authorization.	0.79

The vector branch answers:

Which passages are semantically similar to the query?

It does not inherently guarantee that all intermediate dependencies required for an impact analysis are retrieved.

9 Graph Retrieval Branch

The query is analyzed for known entities and intent.

Example:

```
Entity:
Payments DB

Intent:
Impact analysis

Target:
Checkout
```

Graph traversal can identify a dependency path:

```
Payments DB
  ↓ USED BY
Payment Service
  ↓ CALLED BY
Order Service
  ↓ SUPPORTS
Checkout
```

The relevant entity set is:

```
database_payments
service_payment
service_order
feature_checkout
```

The retrieval layer then maps these entities to supporting chunks.
Example:

Entity	Associated Chunks
Payments DB	C2, C8
Payment Service	C2, C5, C8
Order Service	C3, C9
Checkout	C12, C14

The graph-derived candidate set may therefore be:

C2
C3
C5
C8
C9
C12
C14

10 Candidate Fusion

The vector and graph branches generate partially overlapping evidence.
Example:

Vector candidates:
C2, C8, C12

Graph-derived candidates:
C2, C3, C5, C8, C9, C12, C14

The union is:

C2, C3, C5, C8, C9, C12, C14

This process is better described as **candidate generation and candidate fusion** than as a traditional database join.

11 Candidate Scoring and Reranking

A manual hybrid score may be defined as:

$$Score(c) = \alpha S_v(c) + \beta S_g(c) + \gamma S_e(c) + \delta S_m(c)$$

where:

- $S_v(c)$ is vector semantic similarity;
- $S_g(c)$ is graph relevance, such as path proximity;
- $S_e(c)$ is entity overlap with the query or relevant subgraph;
- $S_m(c)$ is additional metadata relevance.

Example:

$$Score(c) = 0.5 \times SemanticSimilarity + 0.3 \times GraphRelevance + 0.2 \times EntityMatch$$

An alternative is to use a reranker.

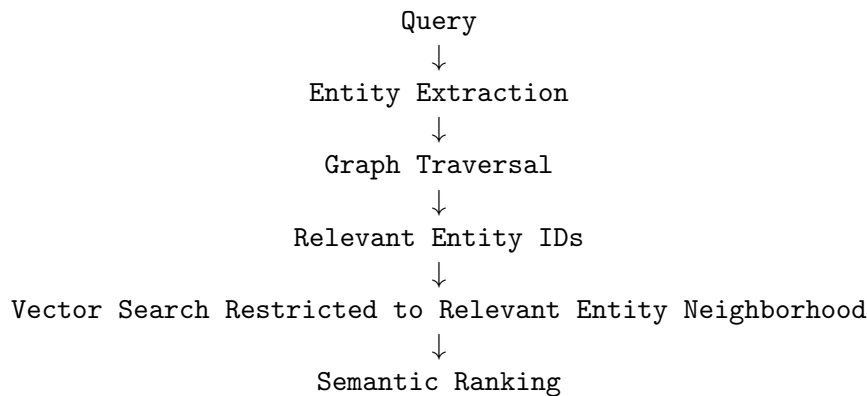
The reranker receives:

```
Query
+
Candidate chunks from vector retrieval
+
Candidate chunks from graph retrieval
```

It produces a relevance ordering based on the complete query and candidate content.
 This architecture avoids forcing all graph and semantic signals into a manually designed formula.

12 Graph-Guided Vector Retrieval

Graph information can also be used before vector ranking.
 The process is:



Example vector query conceptually:

```
Search vectors where:

entity_id IN [
    service_payment,
    service_order,
    feature_checkout
]

ORDER BY semantic similarity
```

This creates a two-stage relevance mechanism:

1. The graph identifies the structurally relevant knowledge neighborhood.
2. The vector store identifies the most semantically relevant evidence within that neighborhood.

13 Retrieval Strategies

13.1 Vector-First

Query → Vector Search → Retrieved Chunks → Entity Identification → Graph Expansion

This pattern is useful when the query does not clearly identify a graph node.
 Example:

What are the major risks during payment processing?

Semantic retrieval can identify candidate entities before graph traversal.

13.2 Graph-First

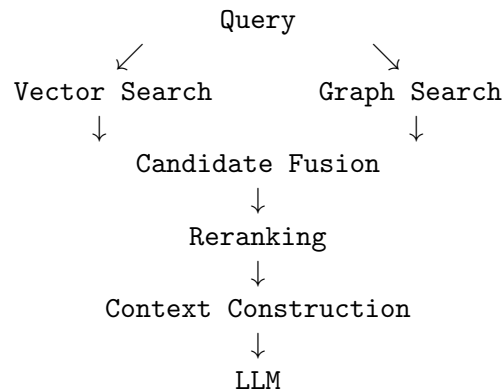
Query → Entity Extraction → Graph Traversal → Entity Neighborhood → Filtered Vector Search

This pattern is suitable when the query contains a clear entity.

Example:

What happens if Payments DB fails?

13.3 Parallel Hybrid Retrieval



This pattern is a suitable general-purpose architecture because neither retrieval mechanism is required to be perfect independently.

14 Graph-Aware Context Construction

The final context should not necessarily consist only of concatenated chunks.

A structured context can include both the graph path and the textual evidence.

Example:

USER QUESTION:
 What happens to Checkout if Payments DB fails?

DEPENDENCY PATH:

Payments DB
 -> USED BY
 Payment Service
 -> CALLED BY
 Order Service
 -> SUPPORTS
 Checkout

SUPPORTING EVIDENCE:

[C2]
 Payment Service uses Payments DB.

[C4]
 Order Service calls Payment Service during checkout.

[C7]
 Order Service supports Checkout.

[Historical Evidence]
 A previous Payments DB outage caused payment authorization failures.

The LLM receives:

1. explicit structural relationships;
2. source-backed textual evidence;
3. optionally ranked historical or operational evidence.

The graph path should normally remain traceable to the underlying source evidence.

15 End-to-End Worked Example

Consider the source material:

Payment Service stores transaction records in Payments DB. Order Service calls Payment Service during checkout. Checkout requires successful payment authorization.

15.1 Step 1: Chunking

C1:
 Payment Service stores transaction records in Payments DB.

C2:
 Order Service calls Payment Service during checkout.

C3:
 Checkout requires successful payment authorization.

15.2 Step 2: Entity Extraction

Payment Service -> Service
 Payments DB -> Database
 Order Service -> Service
 Checkout -> CustomerFeature

15.3 Step 3: Relationship Extraction

The rulebook maps textual evidence to relationships:

"stores transaction records in"
 =>
 Payment Service --USES--> Payments DB

"calls"
 =>
 Order Service --CALLS--> Payment Service

"requires successful payment authorization"
 =>
 Checkout --DEPENDS_ON--> Payment Authorization

Additional domain modeling may connect payment authorization to Payment Service.

15.4 Step 4: Graph Validation

Examples:

```
Service --USES--> Database VALID
Service --CALLS--> Service VALID
CustomerFeature --DEPENDS_ON--> API VALID
```

Invalid relationships are rejected or sent for review.

15.5 Step 5: Vectorization

Each source chunk is embedded independently.

Metadata:

```
C1:
entity_ids = [service_payment, database_payments]

C2:
entity_ids = [service_order, service_payment]

C3:
entity_ids = [feature_checkout, service_payment]
```

15.6 Step 6: User Query

If Payments DB becomes unavailable, what is the likely impact on Checkout?

15.7 Step 7: Plain RAG Retrieval

Plain vector search may retrieve:

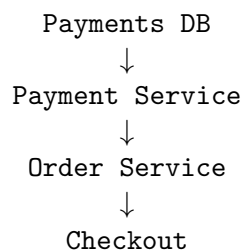
```
C1:
Payment Service stores transaction records in Payments DB.

C3:
Checkout requires successful payment authorization.
```

The intermediate dependency through Payment Service and Order Service may not be retrieved if similarity scoring does not rank the required passage highly enough.

15.8 Step 8: Graph-RAG Retrieval

Graph traversal identifies:



The graph-derived entity set is used to retrieve supporting chunks:

```
C1: Payment Service <-> Payments DB
C2: Order Service <-> Payment Service
C3: Checkout <-> Payment Service
```

The final context contains the complete dependency chain and supporting evidence.

16 Advantages Over Plain RAG

Capability	Plain RAG	Graph-RAG
Primary retrieval signal	Semantic similarity	Semantic similarity plus graph structure
Multi-hop dependencies	Incidental	Explicit graph traversal
Known entity relationships	Usually implicit in text	Explicitly represented
Impact analysis	Depends on retrieving all relevant chunks	Can traverse dependency paths
Candidate expansion	Primarily Top-K similarity	Graph neighborhood expansion
Retrieval filtering	Metadata and similarity filters	Entity, path, neighborhood, and relationship filters
Context structure	Usually chunk concatenation	Chunks plus explicit graph paths
Explainability	Passage-level evidence	Passage-level evidence plus relationship provenance

17 Reference Implementation Plan

17.1 Phase 1: Plain RAG Baseline

Implement:

1. document ingestion;
2. chunking;
3. embedding generation;
4. VectorDB storage;
5. Top-K similarity retrieval;
6. reranking, if required;
7. context assembly;
8. LLM answer generation.

Record baseline metrics for retrieval quality and answer quality.

17.2 Phase 2: Entity and Relationship Extraction

Implement:

1. domain ontology;
2. extraction rulebook;
3. entity extraction;
4. entity normalization and resolution;
5. relationship extraction;

6. confidence generation;
7. schema validation;
8. source provenance capture.

Recommended configuration layout:

```
project/
|
+-- schemas/
| +-- graph_schema.yaml
|
+-- rules/
| +-- relationship_rules.yaml
|
+-- ingestion/
| +-- chunking.py
| +-- entity_extraction.py
| +-- relationship_extraction.py
|
+-- retrieval/
| +-- vector_retriever.py
| +-- graph_retriever.py
| +-- fusion.py
| +-- reranker.py
|
+-- evaluation/
    +-- datasets/
    +-- graph_rag_eval.py
```

17.3 Phase 3: Graph Storage and Provenance

Create graph nodes and relationships.

Store provenance:

```
entity_id
relationship_type
source_chunk_id
source_document_id
confidence
extractor_version
ontology_version
rule_version
created_at
```

17.4 Phase 4: Cross-Store Linkage

Ensure that chunks and graph facts can be connected.

Recommended linkage:

```
Vector chunk
-> chunk_id
-> entity_ids

Graph node
-> entity_id

Graph relationship
-> source_chunk_ids
```


This enables both directions:

Entity → Supporting Chunks

and

Chunk → Mentioned Entities → Graph Neighborhood

17.5 Phase 5: Parallel Retrieval

Implement:

1. semantic vector retrieval;
2. query entity extraction;
3. graph entity resolution;
4. graph traversal;
5. graph-to-chunk mapping;
6. candidate union;
7. deduplication.

17.6 Phase 6: Candidate Reranking

Provide the reranker with:

Query

Candidate source:

- vector
- graph
- both

Chunk content

Optional graph relevance:

- hop distance
- path membership
- entity overlap

Compare:

1. vector-only ranking;
2. graph-only candidate generation;
3. hybrid candidate fusion;
4. hybrid candidate fusion plus reranking.

17.7 Phase 7: Graph-Aware Context Builder

Construct context using sections such as:

Question

Relevant entities

Relationship path

Supporting source evidence

Optional contradictory evidence

Instructions for evidence-grounded answer generation

The context builder should enforce token budgets and avoid including graph information that cannot be traced to an approved or validated knowledge source.

18 Evaluation Framework

A Graph-RAG implementation should not be considered successful merely because a graph database has been added.

Evaluation should compare plain RAG and Graph-RAG on query classes that require graph structure.

Example query classes:

- direct factual lookup;
- entity relationship lookup;
- dependency tracing;
- impact analysis;
- root-cause exploration;
- multi-hop reasoning;
- cross-document relationship discovery.

Suggested measurements:

Metric	Purpose
Context Precision	Percentage of retrieved context that is useful
Context Recall	Ability to retrieve required evidence
Path Accuracy	Correctness of retrieved graph paths
Entity Resolution Accuracy	Correctness of query-to-entity mapping
Relationship Precision	Correctness of extracted edges
Relationship Recall	Coverage of required edges
Answer Faithfulness	Consistency with retrieved evidence
Answer Correctness	Accuracy of the final response
Latency	End-to-end inference performance
Token Cost	Cost of context and generation

The most important comparison is not:

Does Graph-RAG outperform plain RAG on every question?

The relevant comparison is:

Does Graph-RAG improve retrieval and answer quality for questions where entity relationships, dependency paths, or multi-hop structure are necessary?

19 Operational and Governance Considerations

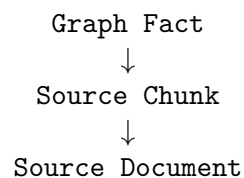
Graph construction introduces additional failure modes:

- incorrect entity extraction;
- duplicate entities;
- incorrect entity resolution;
- hallucinated relationships;
- invalid relationship types;
- outdated graph facts;
- graph traversal over-expansion;
- stale provenance links.

Recommended controls include:

1. ontology versioning;
2. extraction rule versioning;
3. model and prompt version tracking;
4. schema validation;
5. relationship confidence thresholds;
6. source chunk provenance;
7. evaluation datasets for extraction and retrieval;
8. regression testing for ontology and rule changes.

A graph fact should ideally be traceable as:

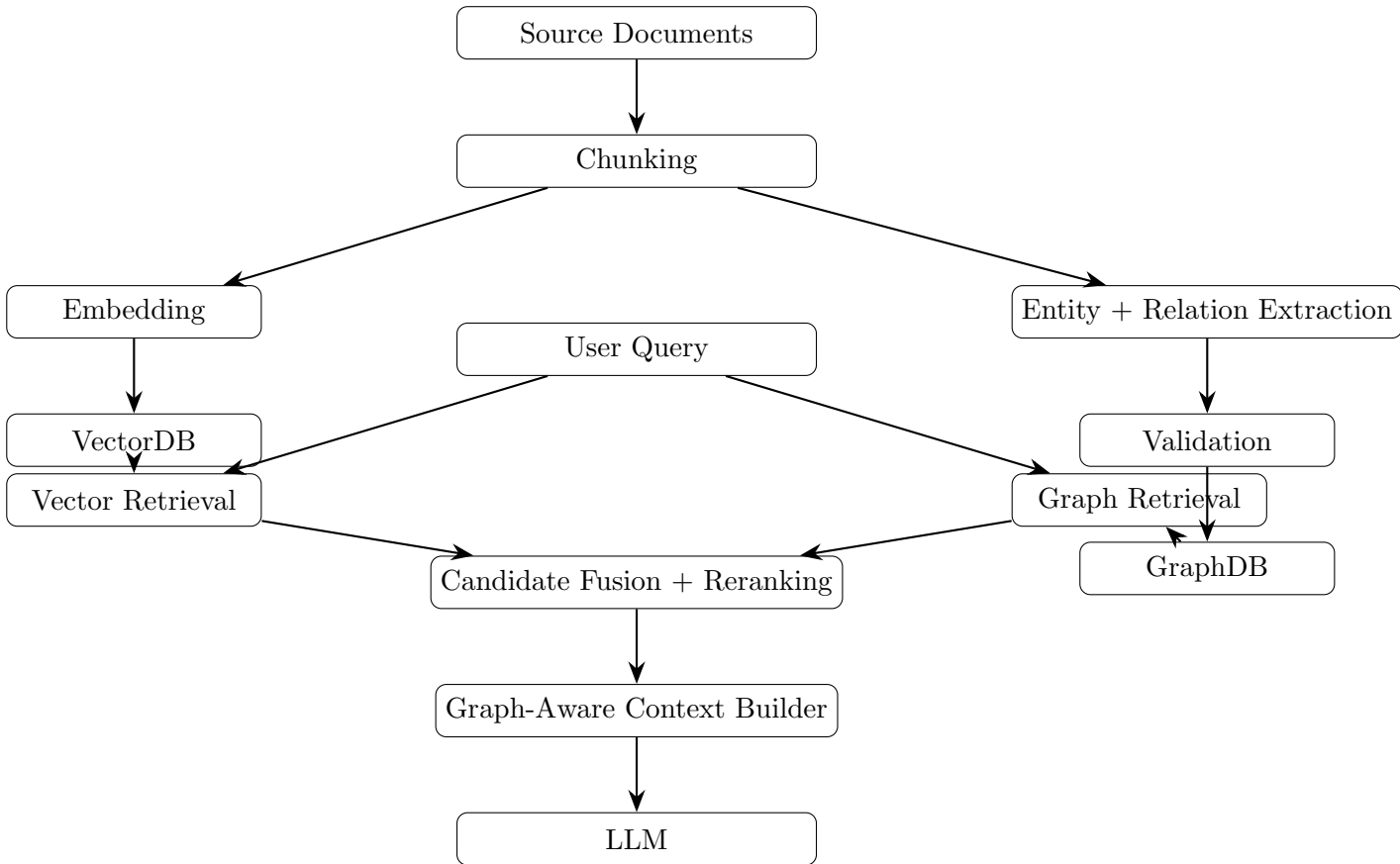


with additional metadata:

Extraction Model + Rule Version + Ontology Version + Confidence

20 Reference Architecture

The complete architecture is:



21 Summary of Design Decisions

The following design choices provide a clean reference implementation:

1. Store chunks and embeddings in the VectorDB.
2. Store entities and validated relationships in the GraphDB.
3. Keep ontology and extraction rules version-controlled outside the GraphDB.
4. Store entity IDs in vector metadata.
5. Store source chunk provenance on graph nodes or relationships.
6. Avoid automatically embedding the entire graph neighborhood into every chunk.
7. Perform graph traversal explicitly at inference time.
8. Support vector-first, graph-first, and parallel retrieval strategies.
9. Use candidate union and deduplication to combine retrieval outputs.
10. Apply reranking to select the strongest evidence.
11. Build final context from both relationship paths and source-backed textual evidence.
12. Evaluate Graph-RAG specifically on multi-hop, dependency, relationship, and impact queries.

The central architectural principle is:

The VectorDB provides semantic evidence retrieval.
The GraphDB provides structural relationship retrieval.
Stable identifiers and provenance connect the two representations.
A retrieval orchestration layer determines how semantic and structural evidence are
fused before context is supplied to the LLM.